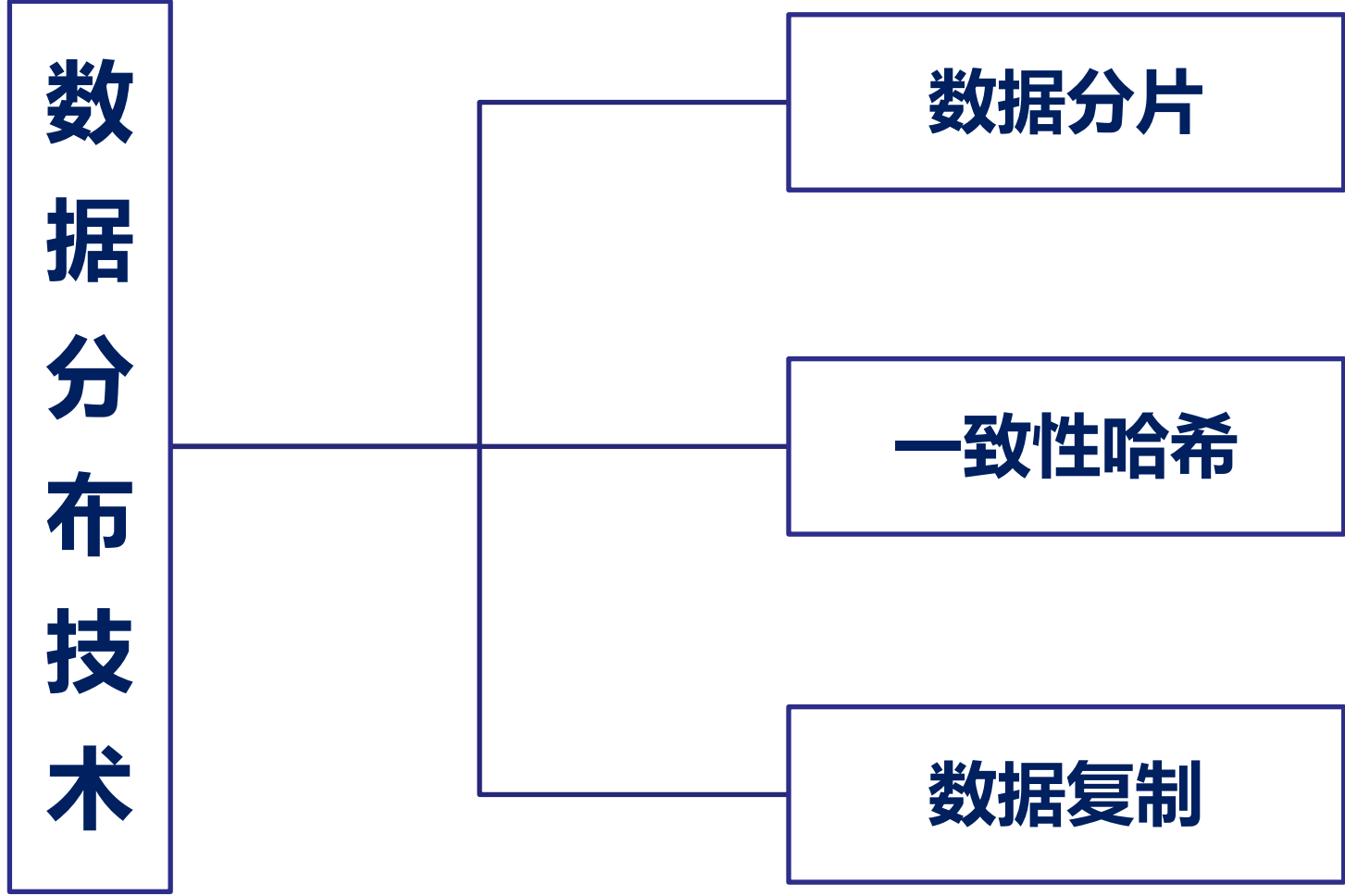


第 13 章 分布式数据库

分布式数据库应用场景

- **电商与金融：**应对高并发交易（如双11）、秒杀、订单洪峰，需强一致性与高可用（如OceanBase、TiDB支撑核心账务）
- **物联网与游戏：**处理海量设备数据实时写入（如智能电网）、全球服架构快速扩容（如王者荣耀）
- **社交与广告：**支持用户动态、好友关系、个性化推荐等大规模数据处理与低延迟访问
- **政务与医疗：**跨部门数据共享（如“一网通办”）、打破数据孤岛，兼顾高并发与隐私保护
- **物流与供应链：**订单轨迹追踪、仓储数据实时同步，多租户隔离与事件驱动架构支撑
- **技术共性：**所有场景均依赖水平扩展、数据分片、多副本容错及一致性模型（强一致或最终一致）

分布式数据库主要服务于高并发在线交易、海量物联网数据、全球多活部署及跨系统数据融合等场景，核心价值在于弹性扩展与高可用



分布式数据存储：水平分片

SalesOrderDetailID	OrderQty	ProductID	SpecialOfferID	UnitPrice	LineTotal
1	1	776	1	2024.994	2024.994
2	3	777	1	2024.994	6074.982
3	1	778	1	2024.994	2024.994
4	1	771	1	2039.994	2039.994

水平分片

水平
分片1

SalesOrderDetailID	OrderQty	ProductID	SpecialOfferID	UnitPrice	LineTotal
1	1	776	1	2024.994	2024.994
2	3	777	1	2024.994	6074.982

水平
分片2

SalesOrderDetailID	OrderQty	ProductID	SpecialOfferID	UnitPrice	LineTotal
3	1	778	1	2024.994	2024.994
4	1	771	1	2039.994	2039.994

分布式数据存储：垂直分片

SalesOrderDetailID	OrderQty	ProductID	SpecialOfferID	UnitPrice	LineTotal
1	1	776	1	2024.994	2024.994
2	3	777	1	2024.994	6074.982
3	1	778	1	2024.994	2024.994
4	1	771	1	2039.994	2039.994

垂直分片

垂直分片1

SalesOrderDetailID	UnitPrice	LineTotal
1	2024.994	2024.994
2	2024.994	6074.982
3	2024.994	2024.994
4	2039.994	2039.994

垂直分片2

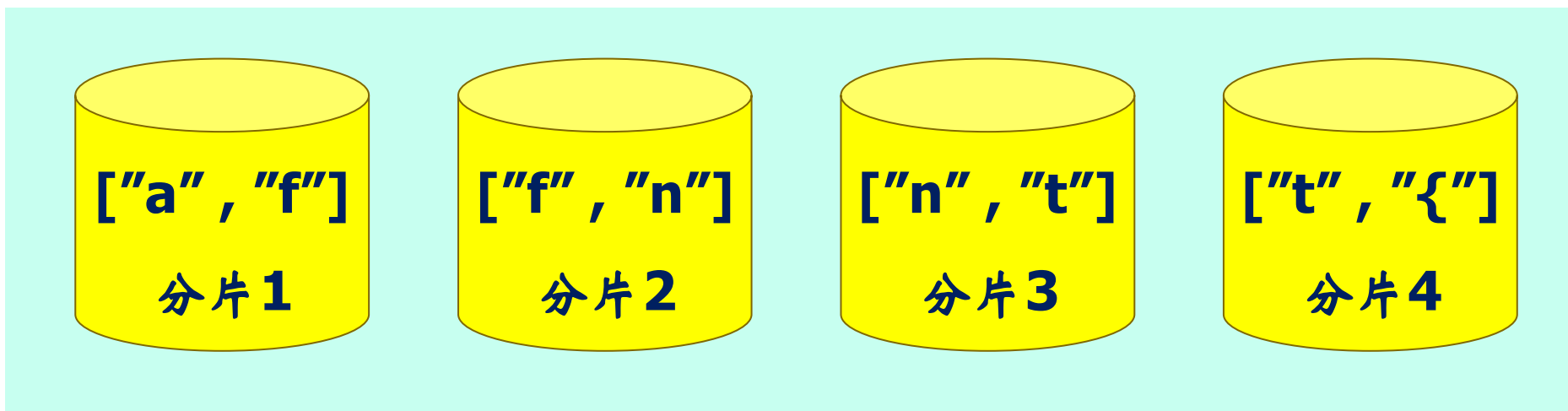
SalesOrderDetailID	OrderQty	ProductID	SpecialOfferID
1	1	776	1
2	3	777	1
3	1	778	1
4	1	771	1

分片过程中解决的主要问题

分片按业务聚合相关数据，各部分放在不同服务器上

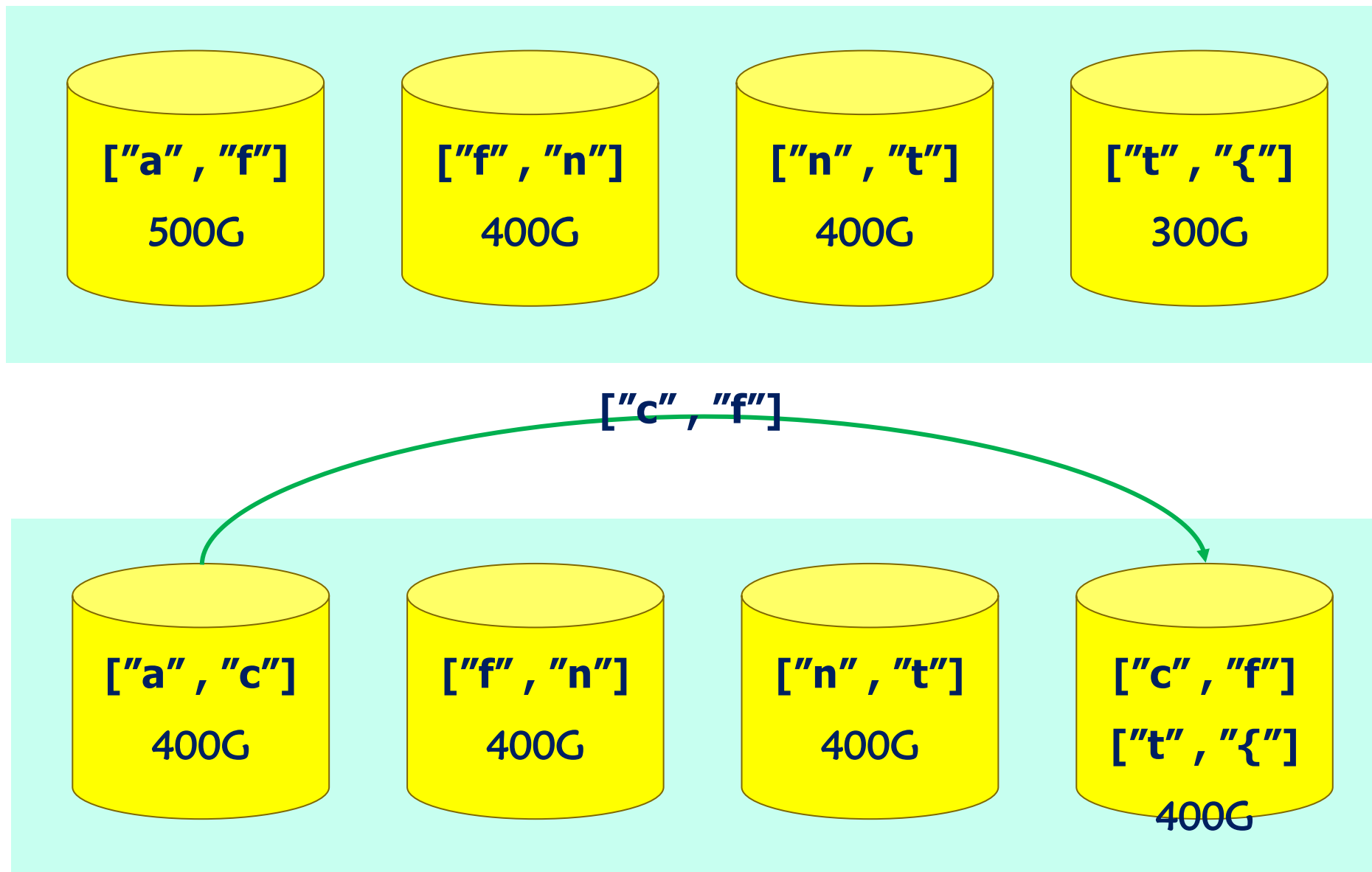
- **划分方式：如何将数据划分到多个节点？**
 - 范围划分，哈希划分
- **路由：查询语句如何正确地转发到相应节点上去执行？**
- **性能：如何动态维持节点间的负载平衡？**

范围划分

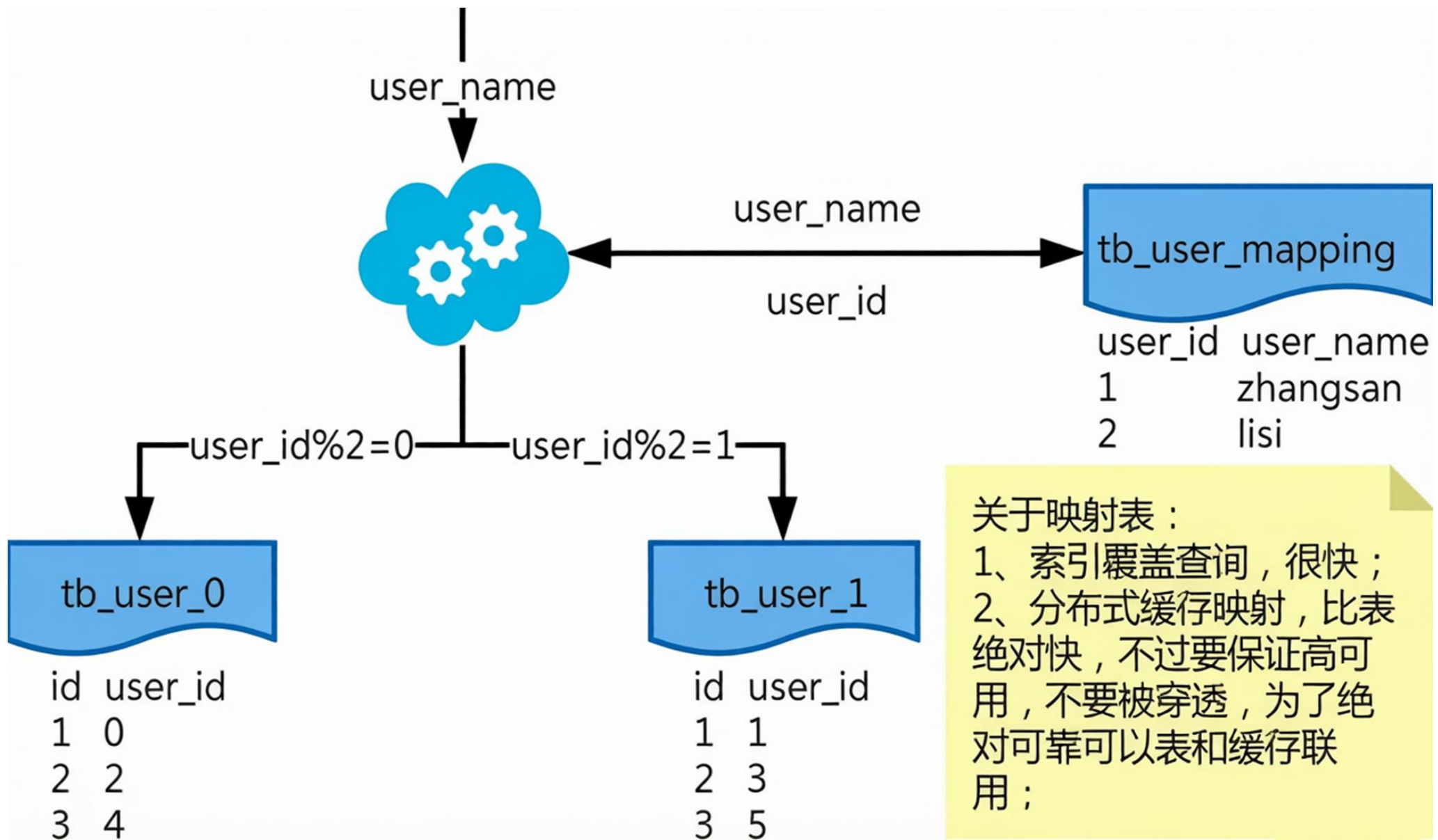


如果一个节点对应一个范围，维护负载均衡需要大量迁移数据

改进：单节点存储多个区间

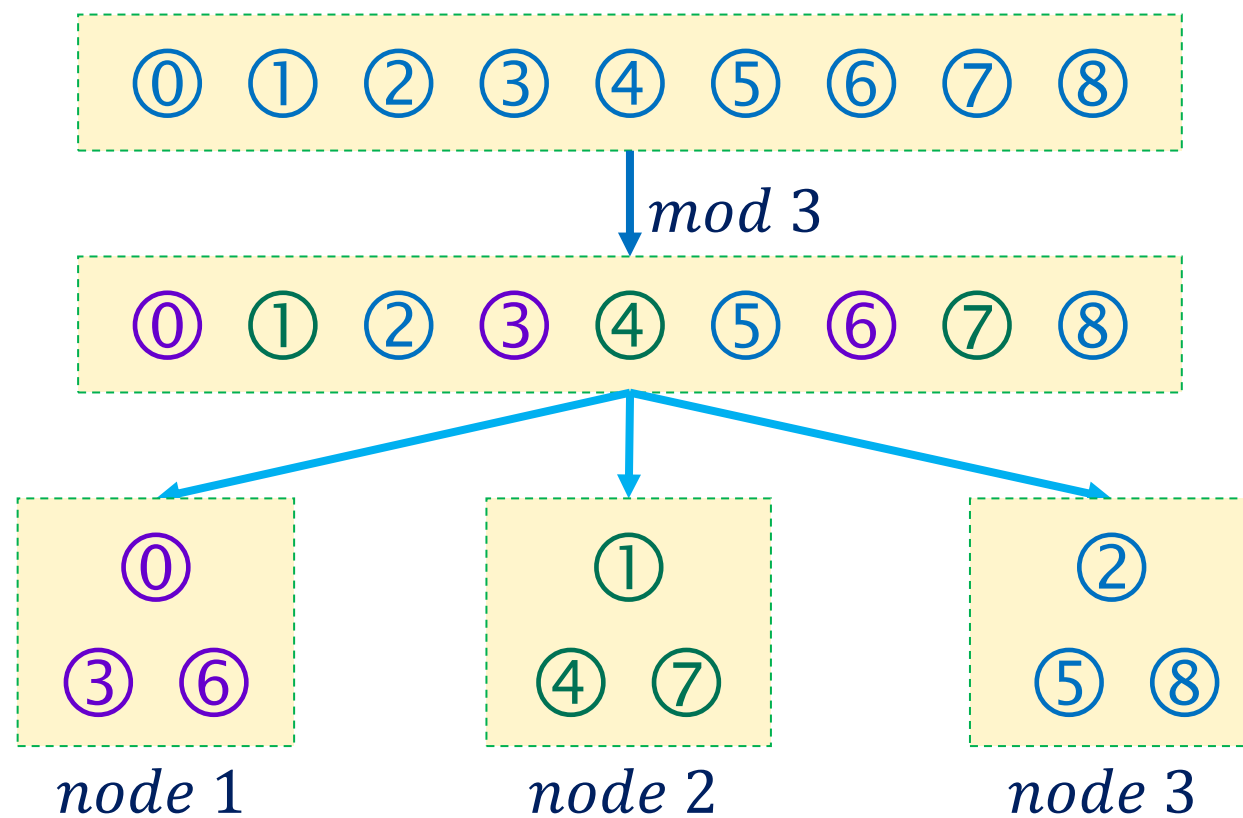


非partition key的查询问题：映射法



哈希划分

$$n = t.Key \bmod N \Rightarrow n = \text{hash}(t.Key) \bmod N$$



适合点查询和全量扫描

哈希划分中的缓存失效问题

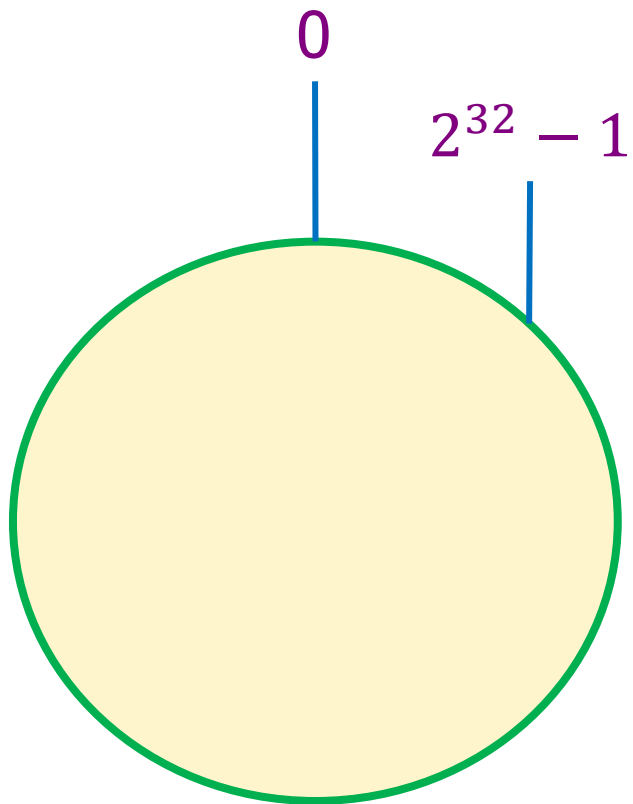
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
mod 4	0	1	2	3	0	1	2	3	0	1	2	3	0	1	2	3	0	1	2	3

增加一个节点后

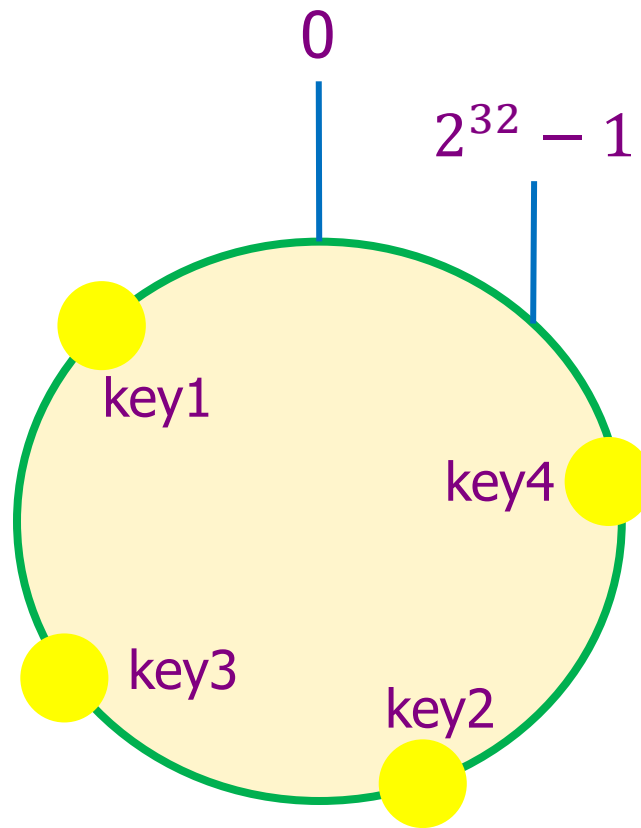
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
mod 5	0	1	2	3	4	0	1	2	3	4	0	1	2	3	4	0	1	2	3	4

哈希算法的单调性：如果已经有一些内容哈希到现有节点，当新节点加入到系统中时，要保证原有已分配的内容仍被映射到旧节点，而不会映射到新的节点

一致性哈希算法

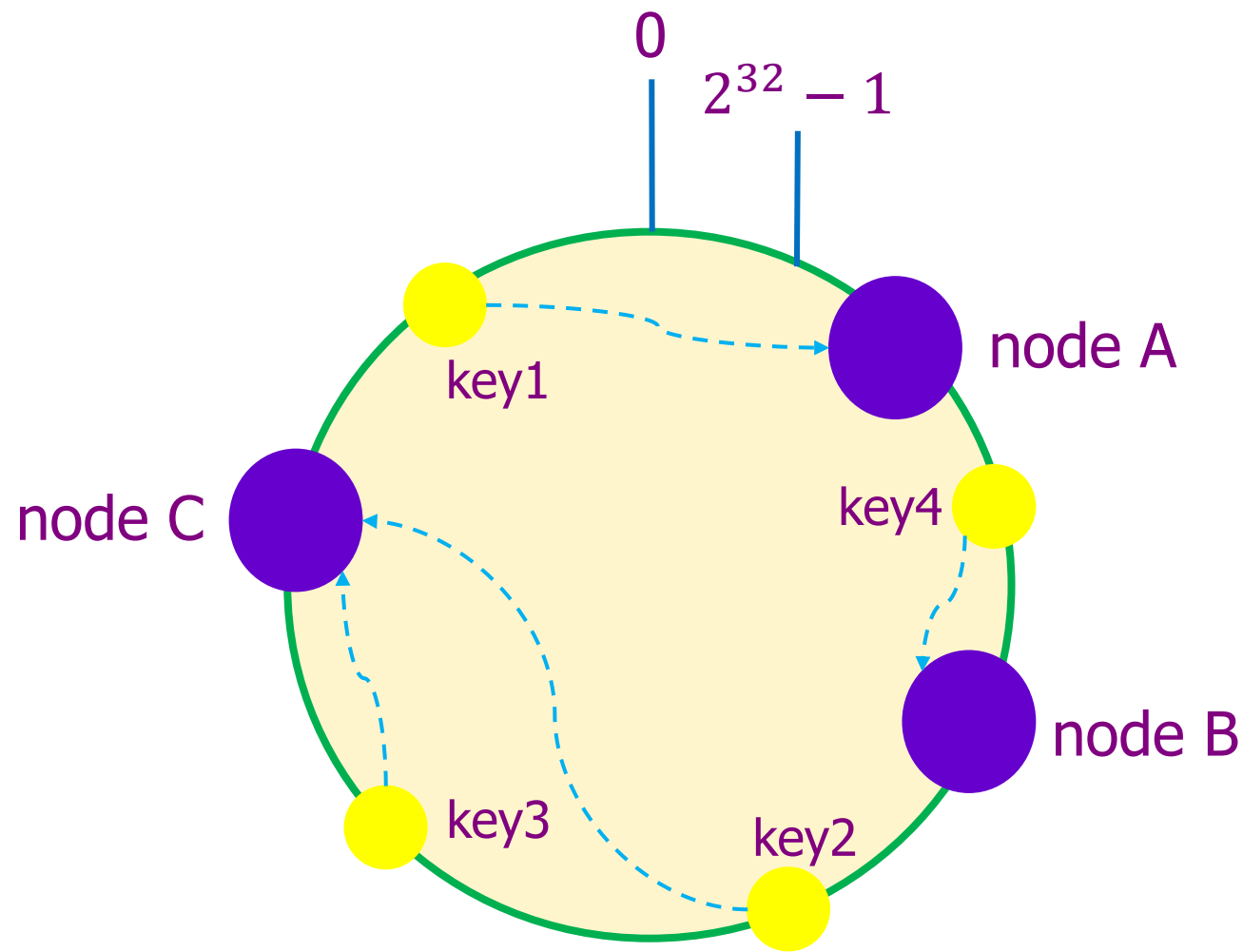


环形hash空间

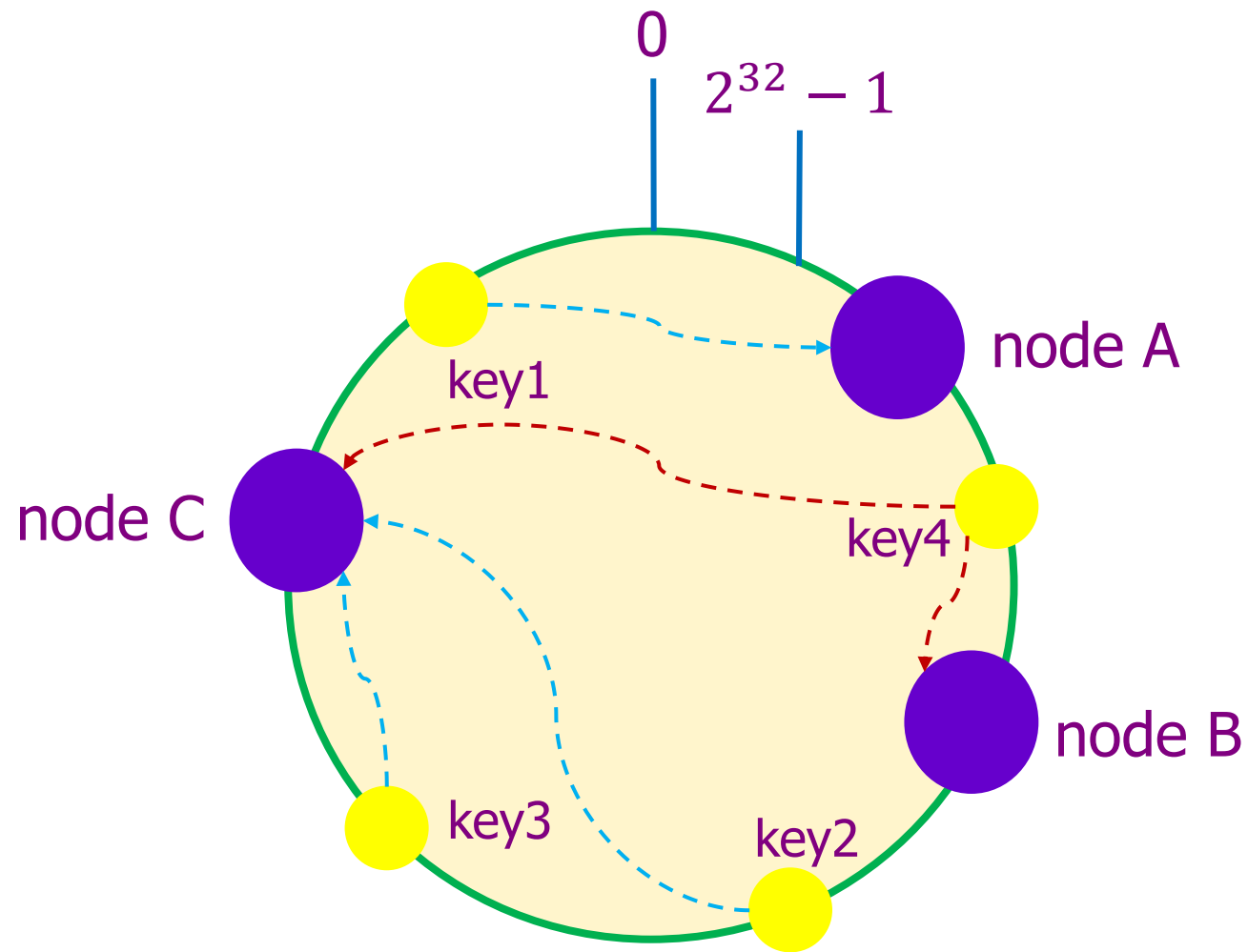


4个对象的key值分布

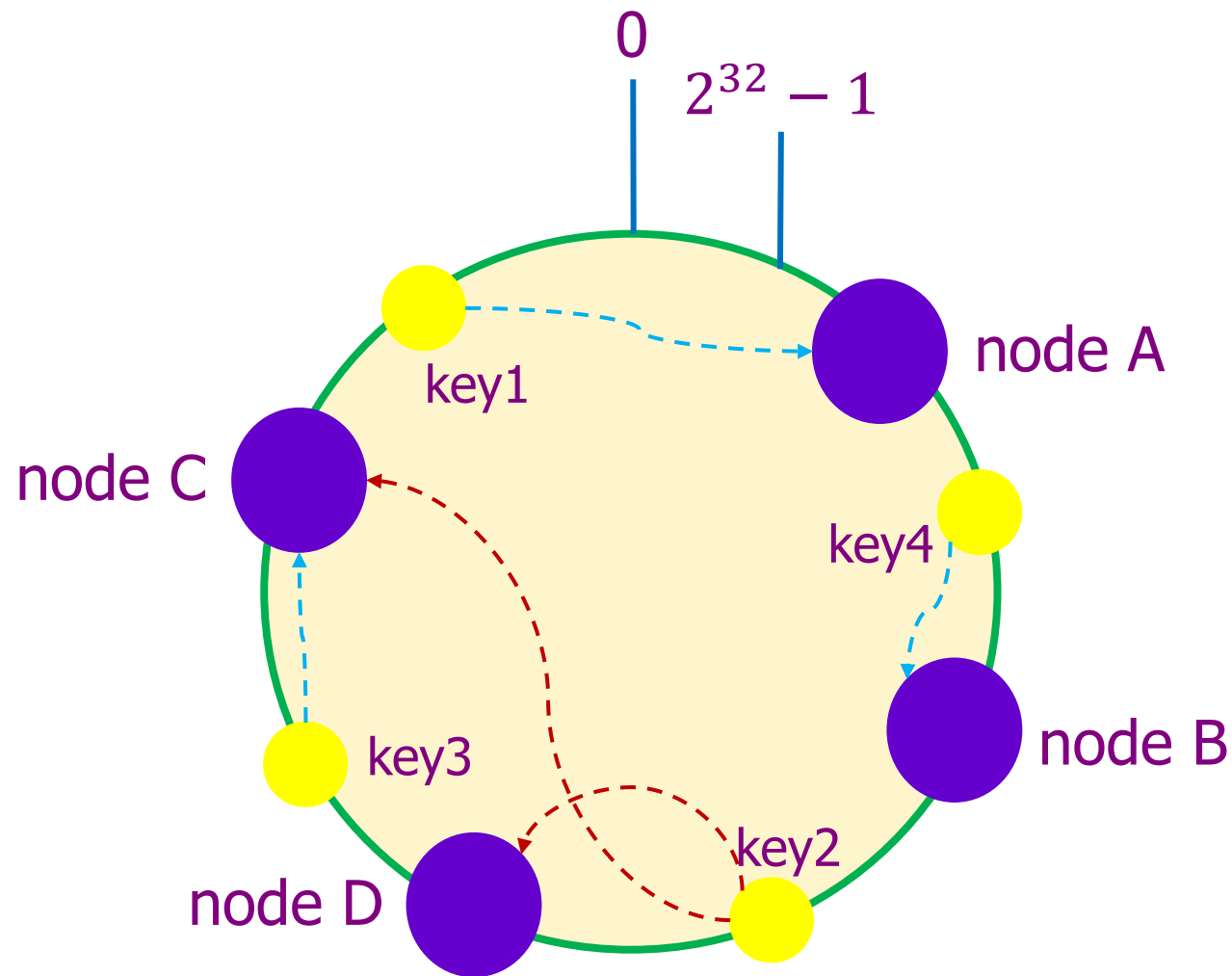
节点和对象的key值分布



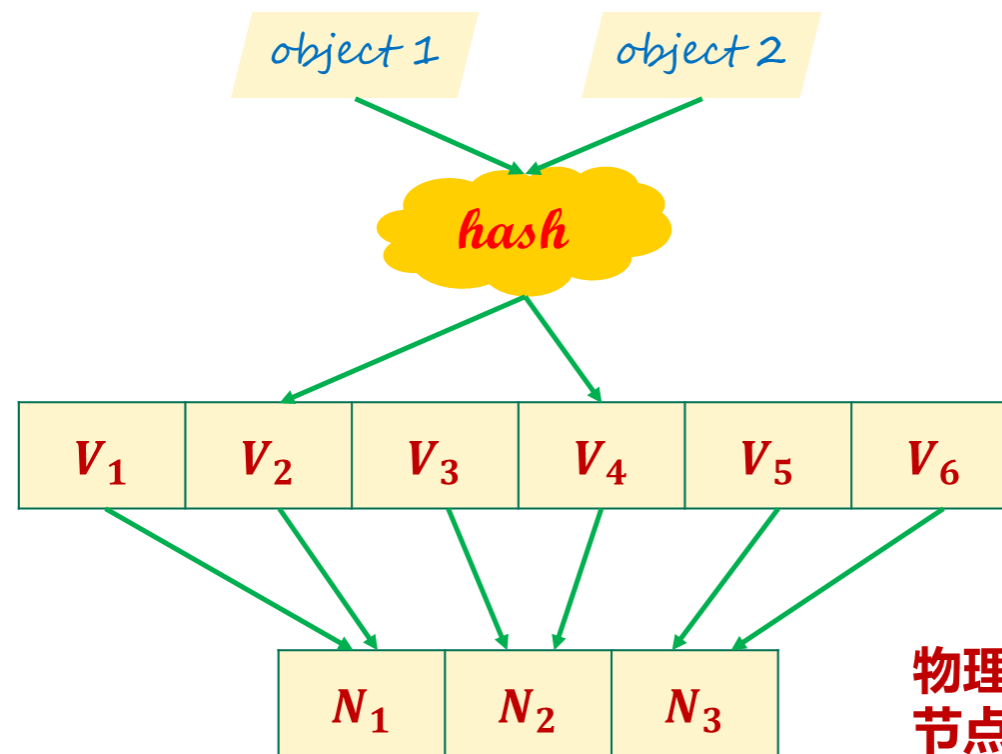
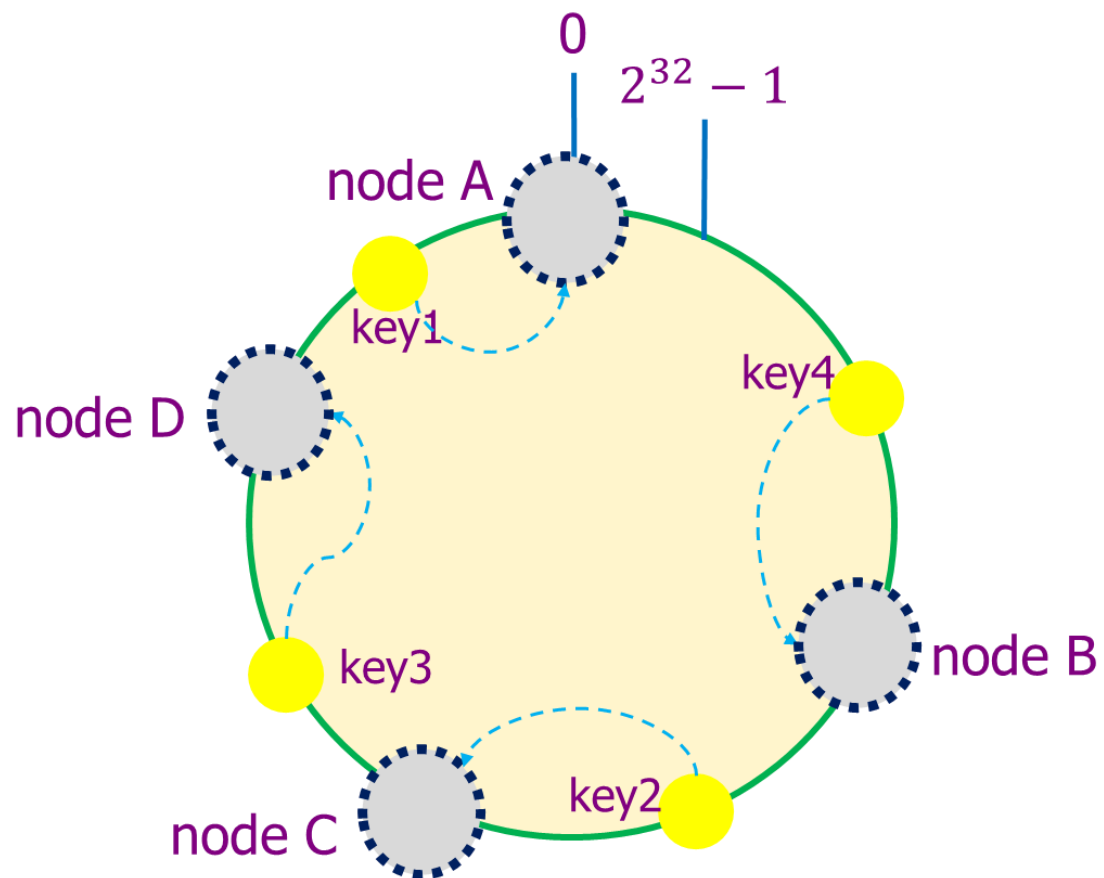
节点被移除后的key值分布



添加节点后的key值分布



一致性哈希算法中的虚拟节点



分布式数据存储：复制

节点1

SalesOrderDetailID	OrderQty	ProductID	SpecialOfferID	UnitPrice	LineTotal
1	1	776	1	2024.994	2024.994
2	3	777	1	2024.994	6074.982
3	1	778	1	2024.994	2024.994
4	1	771	1	2039.994	2039.994



节点2

SalesOrderDetailID	OrderQty	ProductID	SpecialOfferID	UnitPrice	LineTotal
1	1	776	1	2024.994	2024.994
2	3	777	1	2024.994	6074.982
3	1	778	1	2024.994	2024.994
4	1	771	1	2039.994	2039.994

为什么需要复制？

分布式系统的根本挑战：硬件故障常态化



问题陈述：故障是常态，而非意外

- 在成百上千台普通服务器组成的集群中，磁盘损坏、节点宕机、网络中断不是偶发的“小概率事件”，而是需要时刻应对的常态。

数据佐证：惊人的故障率

- 单块机械硬盘年故障率(AFR)约为 1%-3%。对于拥有1000个节点的集群，这意味着平均每周就会发生 1-2 次严重的硬件故障。

单节点风险：不可接受的后果

- 如果数据只存一份，任何单点故障都可能导致关键业务数据永久丢失，服务被迫中断。

一个算例：如果集群中没有复制机制？

MTTF (平均失效时间): 无故障运行的平均时间

AFR (年故障率): $AFR = 1 / (MTTF \div 24 \div 365)$

单盘算例: 某 SSD 标称 MTTF=200 万小时, 计算得 $AFR \approx 0.44\%$

无副本集群模型: 串联系统, 任意节点故障都会导致数据丢失

集群 MTTF: $MTTF_{\text{集群}} = MTTF_{\text{磁盘}} / n$ (n 为节点数)

集群 AFR: $AFR_{\text{集群}} \approx n \times AFR_{\text{磁盘}}$

集群规模	单盘 MTTF	集群 MTTF	集群 AFR	预计故障间隔
10 节点	200 万小时	20 万小时	4.38%	约 22.8 年
100 节点	200 万小时	2 万小时	43.8%	约 2.28 年
1000 节点	200 万小时	2000 小时	438%	约 83 天

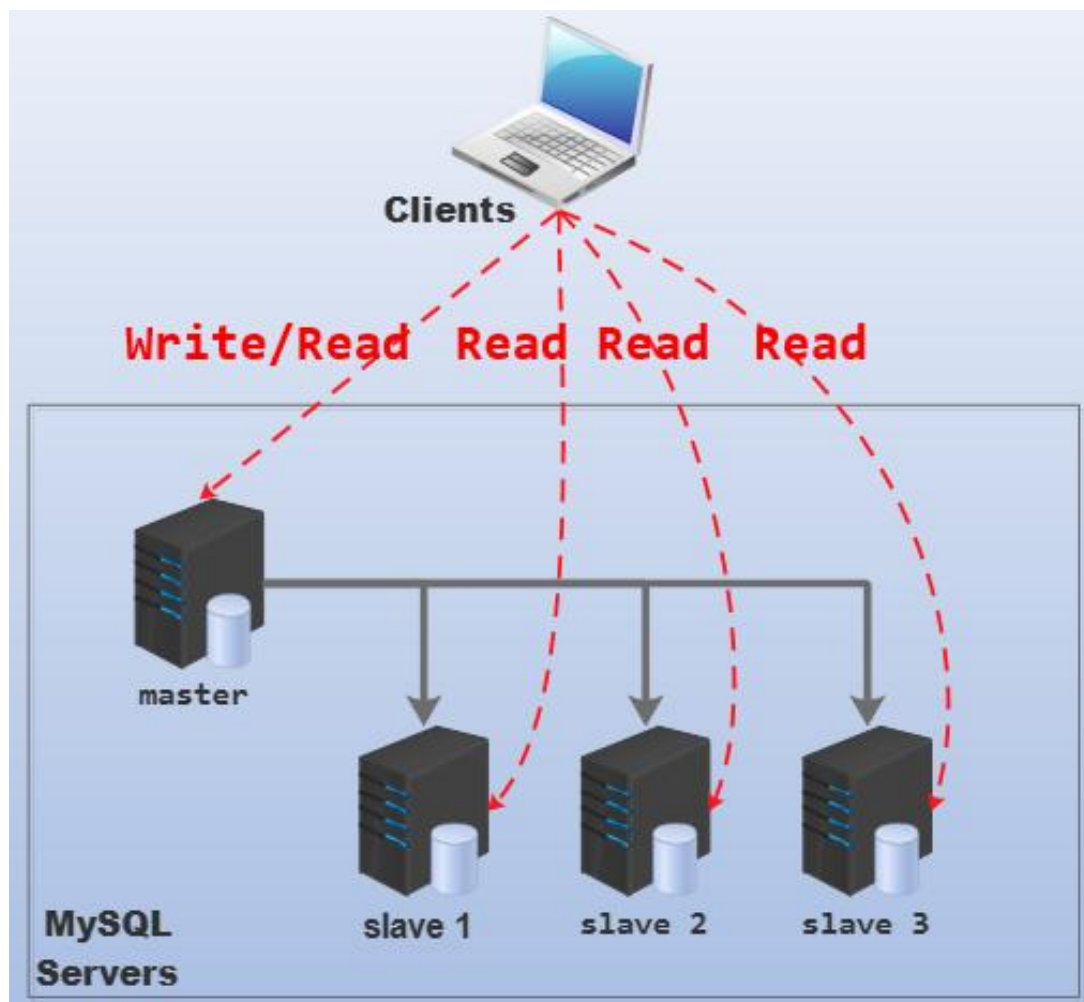
通过多复本机制提高存储系统的可靠性

$$MTTF_{m\text{复本}} = MTTF_{\text{磁盘}} \times H_m$$

其中 H_m 是第 m 个调和数： $H_m = 1 + 1/2 + 1/3 + \dots + 1/m$

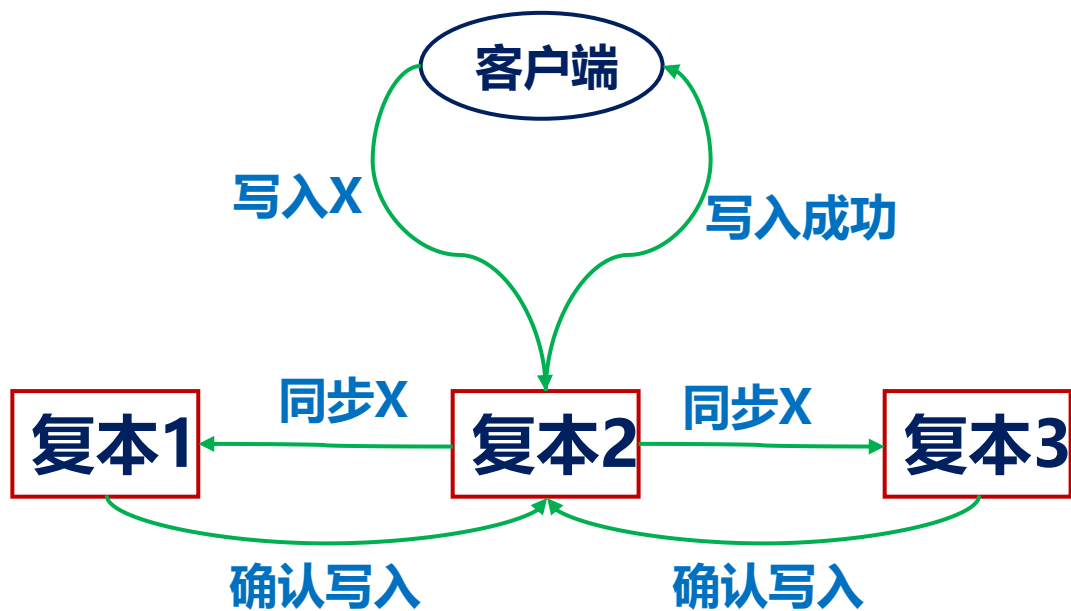
方案	集群规模	单盘MTTF	集群/数据MTTF	提升倍数
无副本 (n=10)	10节点	200万小时	20万小时 (约22.8年)	基准
无副本 (n=100)	100节点	200万小时	2万小时 (约2.28年)	—
无副本 (n=1000)	1000节点	200万小时	2000小时 (约83天)	—
2副本 (m=2)	单个数据块	200万小时	300万小时 (约342年)	单盘1.5倍
3副本 (m=3)	单个数据块	200万小时	367万小时 (约418年)	单盘1.83倍
5副本 (m=5)	单个数据块	200万小时	457万小时 (约521年)	单盘2.28倍

复制的优点



- **数据可靠性**: 几乎所有的分布式存储都必须用某种冗余的方式在廉价硬件的基础上搭建高可靠的存储, 而冗余的基础就是多副本策略
- **数据可用性**: 确保用户在任何时候都能访问到数据, 即使部分节点或网络出现问题。当某个节点不可用时, 请求会被透明地重定向到其他健康副本
- **读取性能**: 通过增加数据副本的数量, 利用负载均衡和数据本地性机制, 可以分散读取负载, 有效提高系统的并发读取能力

多复本的写入过程



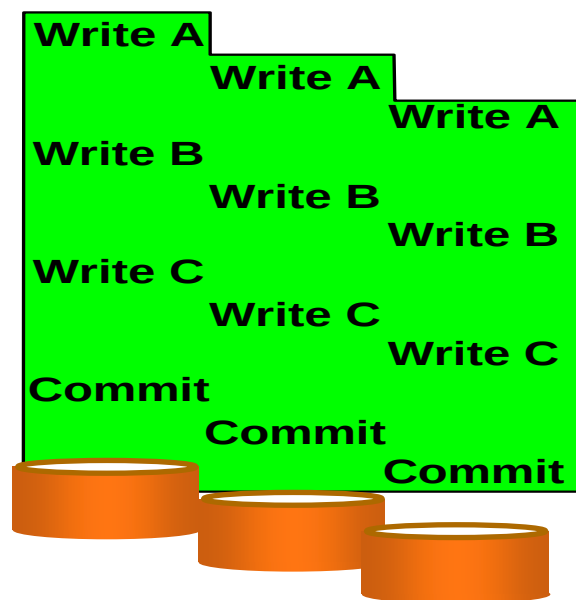
问题一：复本之间的更新是同步还是异步完成的？

问题二：哪些复本可以接受更新请求？

数据复制的更新传播策略

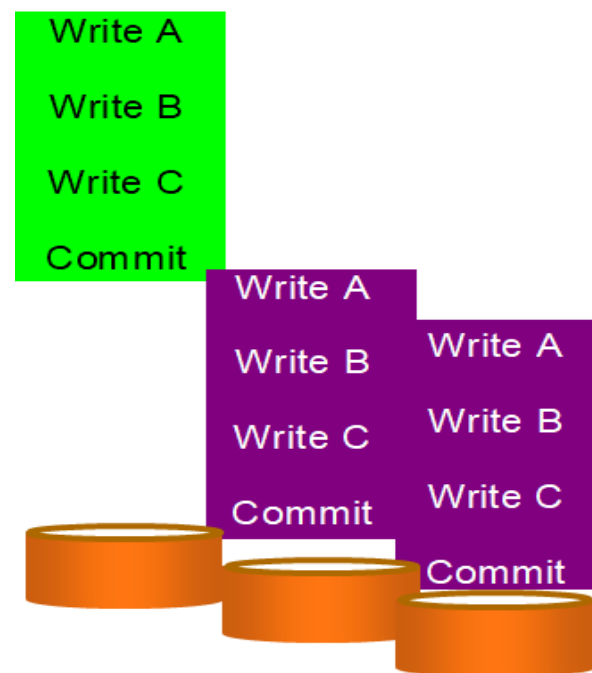
紧密复制(Eager Replication)

- 将所有结点上的复本更新作为一个原子事务的一部分，所有结点上的复本严格同步



松散复制(Lazy Replication)

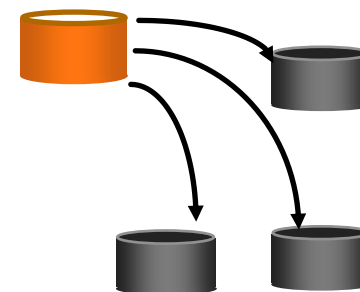
- 更新事务提交之后，异步地将更新传向其它结点



数据复制的更新控制策略

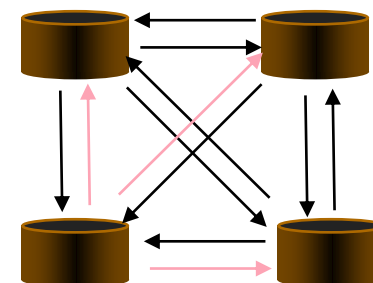
主方式(master)

- 每一对象都拥有一个主结点，只有主结点能够更新对象的主拷贝，其它复本是只读的。其它欲更新该对象的结点请求主结点完成更新，由主结点将更新传向其它结点



群方式(group)

- 任何拥有某数据项拷贝的结点都可以更新该拷贝，称之为随处更新(update anywhere)，并将更新操作广播到其它结点

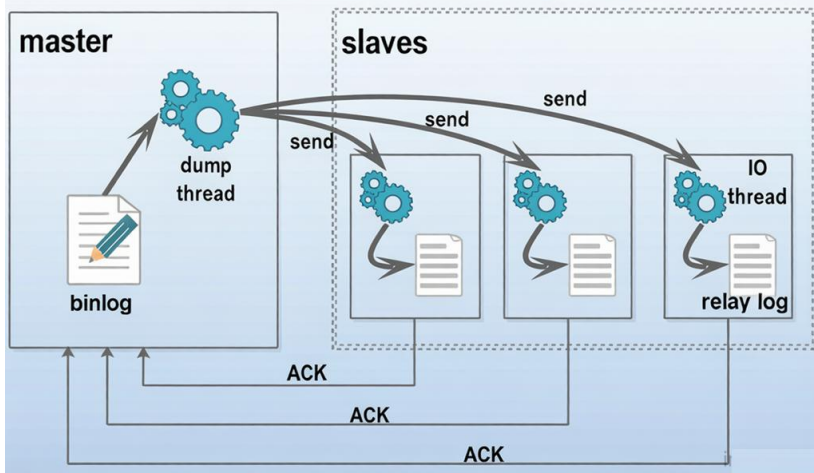


结合不同更新策略的数据复制类型

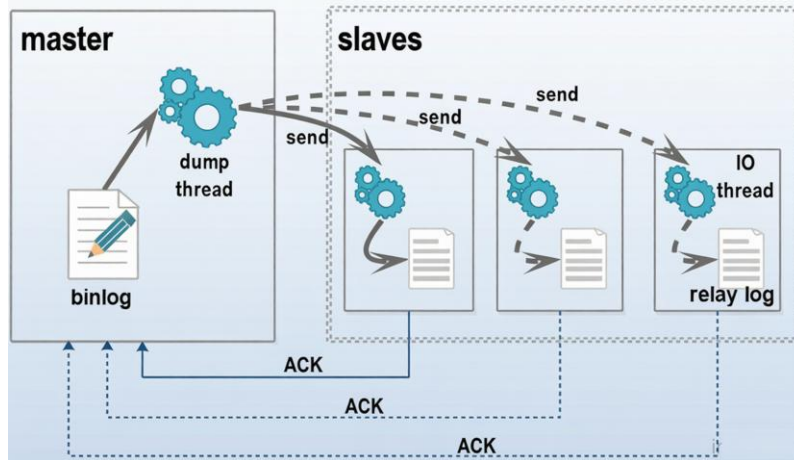
传播方式 拥有方式	松散	紧密
群	N个事务 N个对象拥有者	1个事务 N个对象拥有者
主	N个事务 1个对象拥有者	1个事务 1个对象拥有者

MySQL复制类型

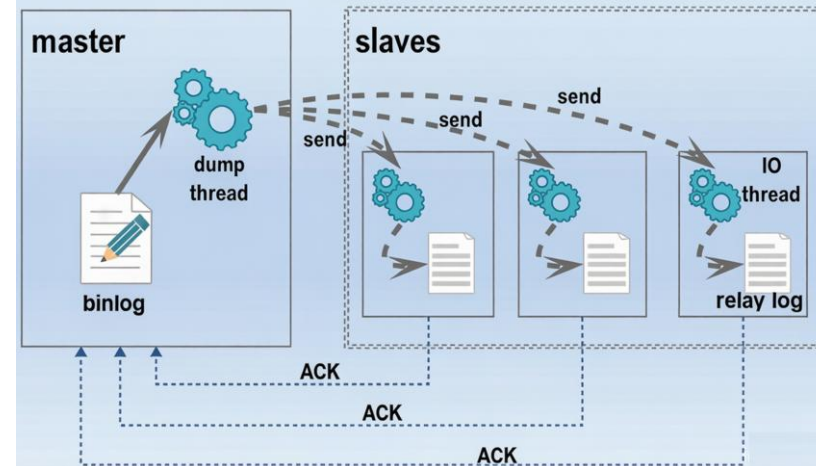
同步复制：
所有slave都写完relay log

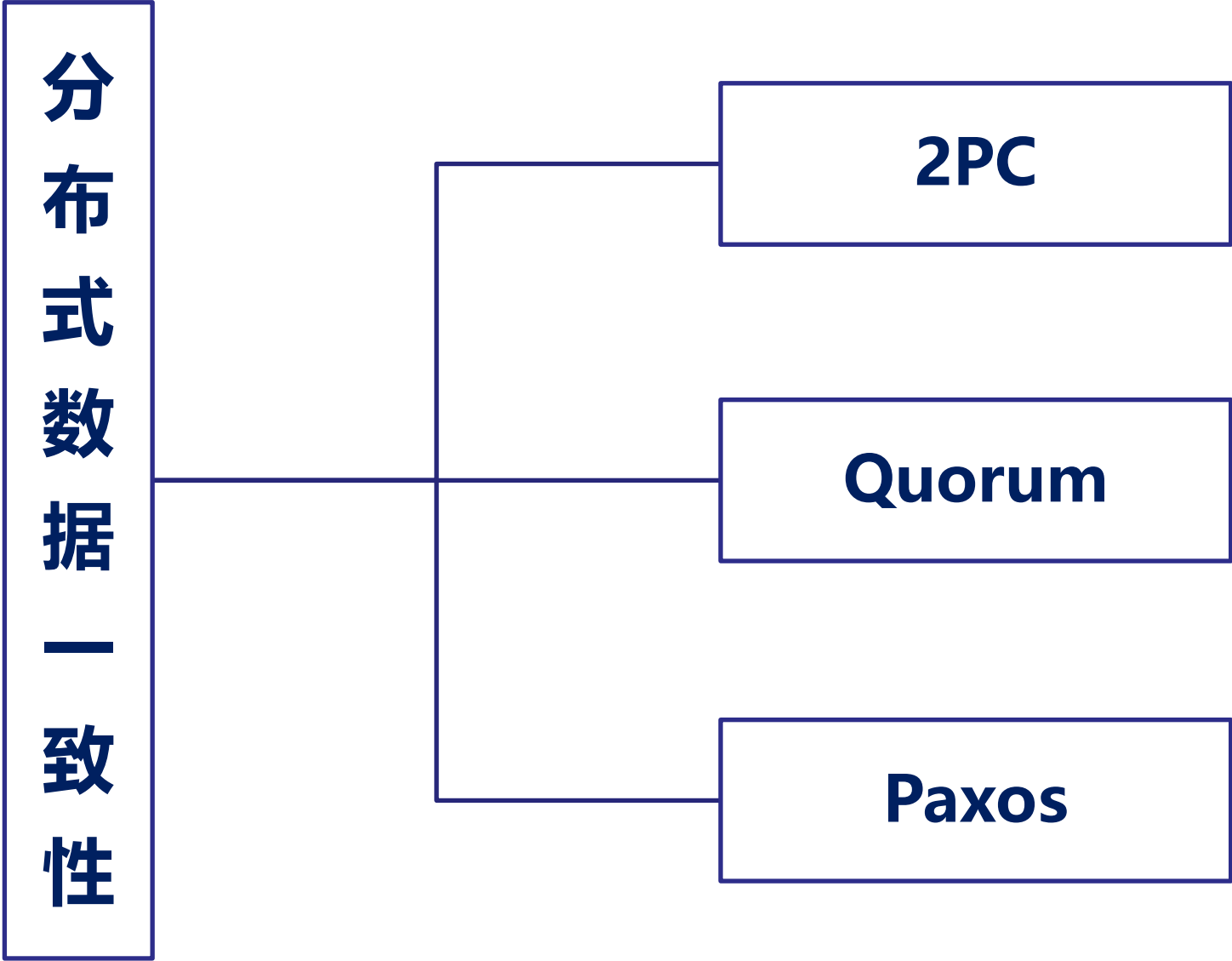


半同步复制：
等待一个slave写完relay log

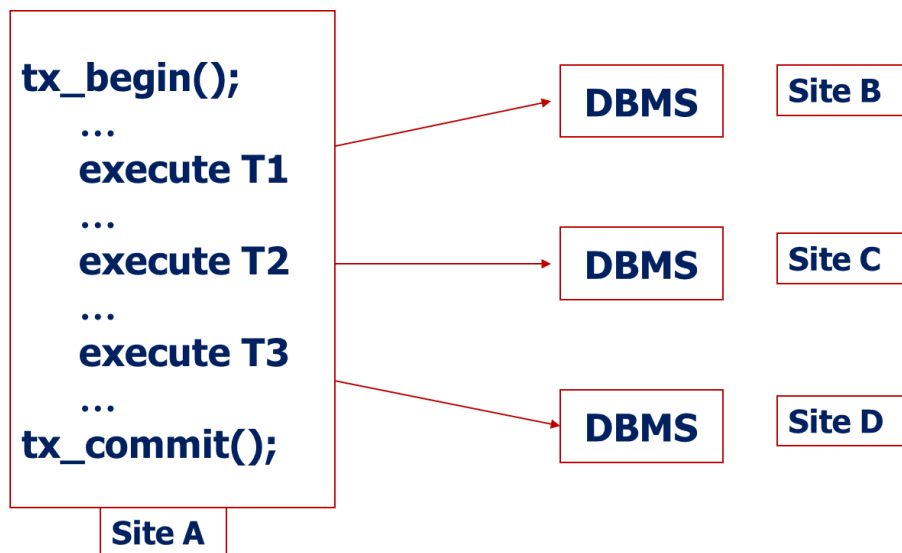


异步复制：直接返回





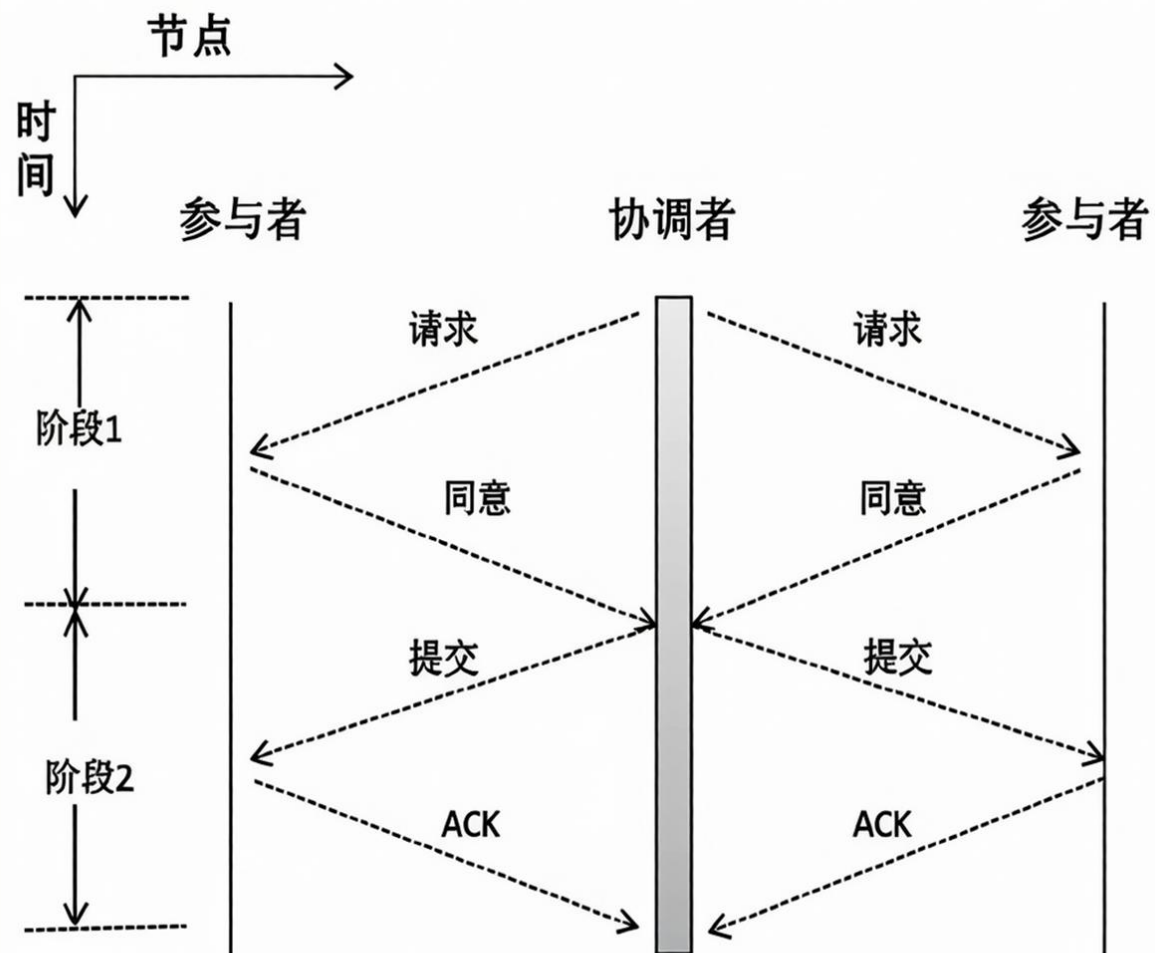
分布式事务的两阶段提交：2PC



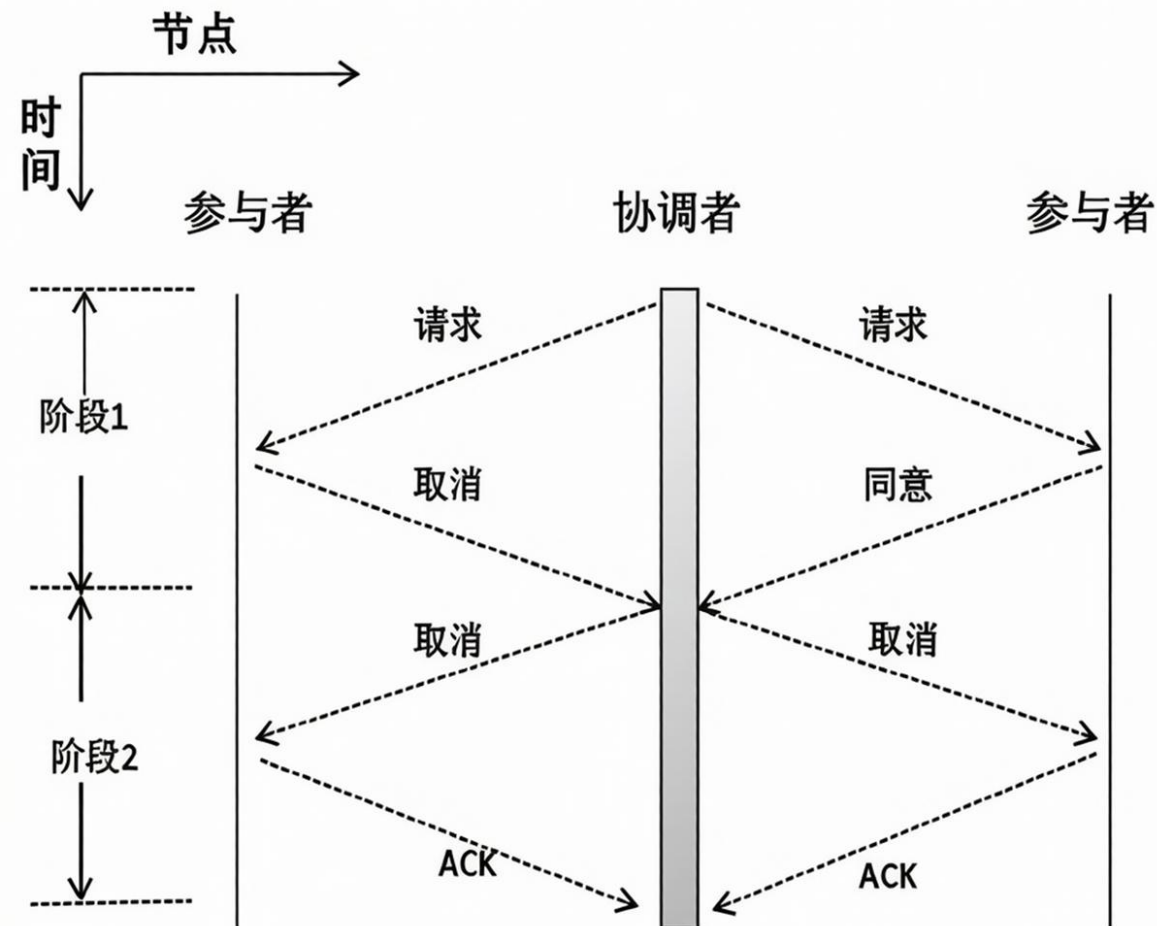
- 引入一个作为协调者的组件来统一掌控所有节点(称作参与者)的操作结果并最终指示这些节点是否要把操作结果进行真正的提交(比如将更新后的数据写入磁盘等等)
- 二阶段提交的算法思路可以概括为：参与者将操作成败通知协调者，再由协调者根据所有参与者的反馈情报决定各参与者是否要提交操作还是中止操作



两阶段提交：2PC



(a) 成功



(b) 失败

两阶段提交的问题

问题类型	核心描述	影响后果
同步阻塞	协调者发出请求后，参与者持有资源锁并等待最终指令，期间无法处理其他事务。	系统并发能力大幅下降，吞吐量受限
单点故障	协调者是关键节点，一旦宕机，所有参与者将处于“悬而未决”状态，无法推进事务。	事务长时间挂起，直至协调者恢复
数据不一致	第二阶段（提交/中止）因网络异常或节点宕机，导致部分参与者执行提交，部分未执行。	破坏分布式事务的原子性
协调者不确定性	协调者发出提交指令后宕机，重启后无法确认指令是否已被所有参与者成功执行。	需要额外的日志恢复或人工介入
脑裂	网络分区时，可能出现多个“协调者”分别控制不同参与者	事务执行逻辑冲突，数据完整性受损

多副本一致性：Quorum系统NRW策略

对于分布式数据系统，指定如下参数：

N — 数据复制的份数

W — 更新数据是需要保证写完成的节点数

R — 读取数据的时候需要读取的节点数

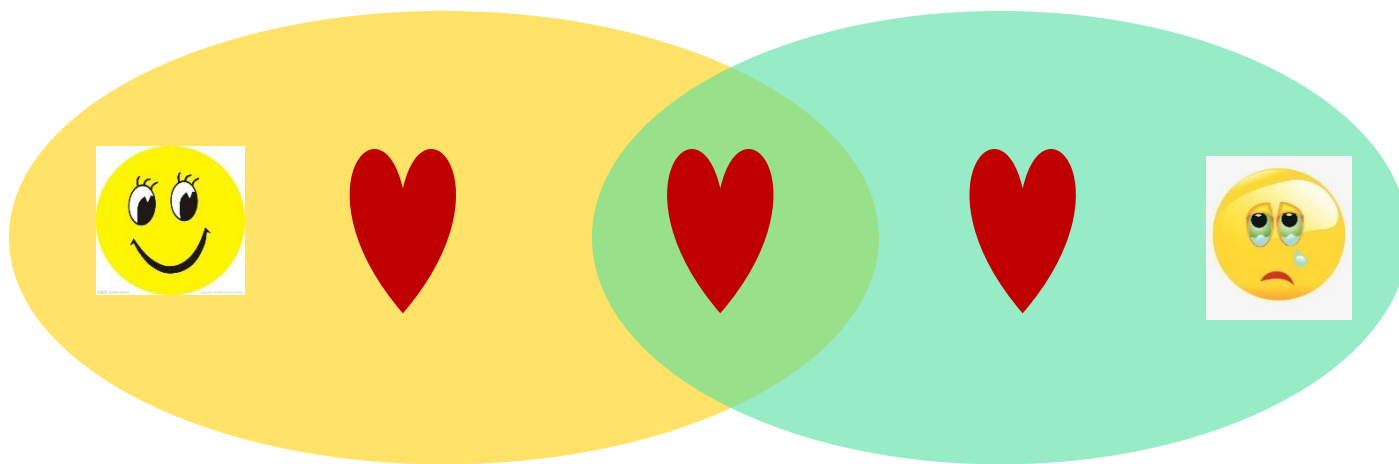
Quorum ：对WARO条件进行松弛，对读写服务可用性做折中

Quorum系统NRW策略

若 $W + R > N$, 读写节点重叠, 则是强一致性

一般设定 $R + W = N + 1$, 这是保证强一致性的最小设定

例如 $N = 3, W = 2, R = 2$



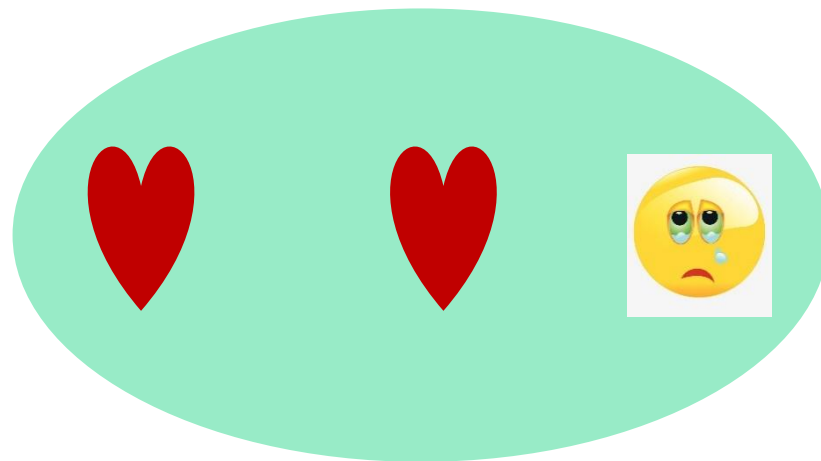
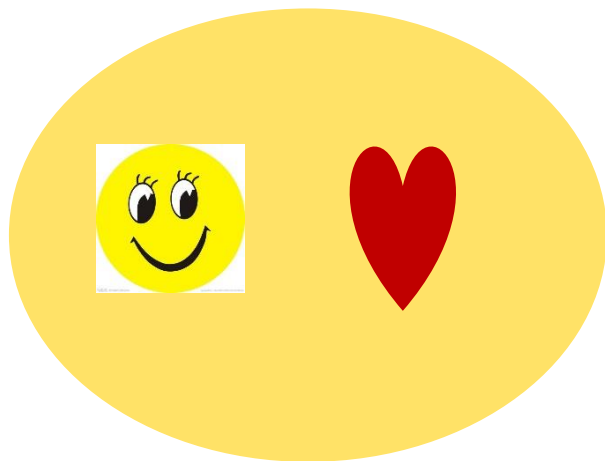
Quorum系统NRW策略

若 $W + R \leq N$, 则是弱一致性

例如 $N = 3, W = 2, R = 1$

此时如果读的是备库，就可能无法读取主库

已经更新过的数据，所以是弱一致性



Quorum系统的几个特例

若 $W = 1, R = N$

系统对写操作的性能有较高要求，但读操作比较慢

若N个节点之一发生故障，那么读操作将不能完成

若 $R = 1, W = N$

系统对读操作的性能有较高要求，但写操作比较慢

若N个节点之一发生故障，那么写操作将不能完成

若 $R = W = \frac{N}{2} + 1$

系统在读写性能之间取得了平衡

多副本一致性：Paxos算法

- ◆ 虚构的Paxos议会
- ◆ 第一次投稿
- ◆ 第二次投稿
- ◆ 第三次投稿



2013年图灵奖得主
微软科学家
Leslie Lamport

Paxos Made Simple

Leslie Lamport

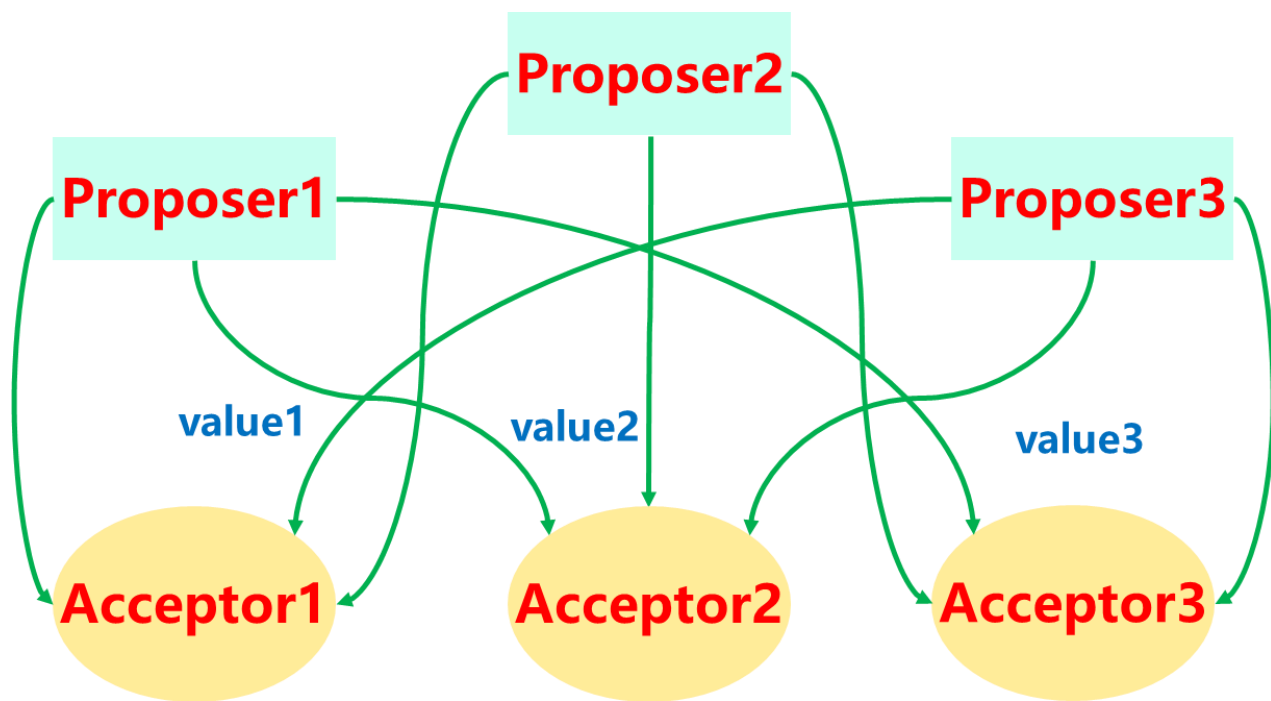
01 Nov 2001

Abstract

The Paxos algorithm, when presented in plain English, is very simple.

Paxos算法中的角色

- **Proposer**提出提案 (Proposal), Proposal包括提案编号(Proposal ID) 和提议的值 (Value)
- **Acceptor**参与决策, 回应Proposers的提案
- 若Proposal获得多数Acceptors的接受, 则称该Proposal被批准



如何保证在多个Acceptor和Proposer的情况下选定一个value

Paxos算法步骤：整体流程

第一阶段 Prepare阶段

Proposer 向 Acceptors 发出 **Prepare** 请求，Acceptors 针对收到的 Prepare 请求进行 **Promise** 承诺



第二阶段 Accept阶段

Proposer 收到多数 Acceptors 承诺的 Promise 后，向 Acceptors 发出 **Accept** 请求，Acceptors 针对收到的 Accept 请求进行 **批准**



第三阶段 Learn阶段

Proposer 在收到多数 Acceptors 的 Accept 之后，标志着本次 Accept 成功，决议形成，将形成的决议发送给所有 Learners

Paxos算法演示

Proposer
(N, V)

Acceptor
(ResN, AcceptN, AcceptV)

Prepare(N), N递增, 不重复

Prepare请求

若 $N \leq \text{ResN}$, 不响应或响应 <error>
若 $N > \text{ResN}$, 令 $\text{ResN} = N$,
响应 [Pok, AcceptN, AcceptV]

Response

若收到超半数的Pok, 发出Accept[N, V]
若Pok未超半数, 重新获取N, 发起Prepare请求

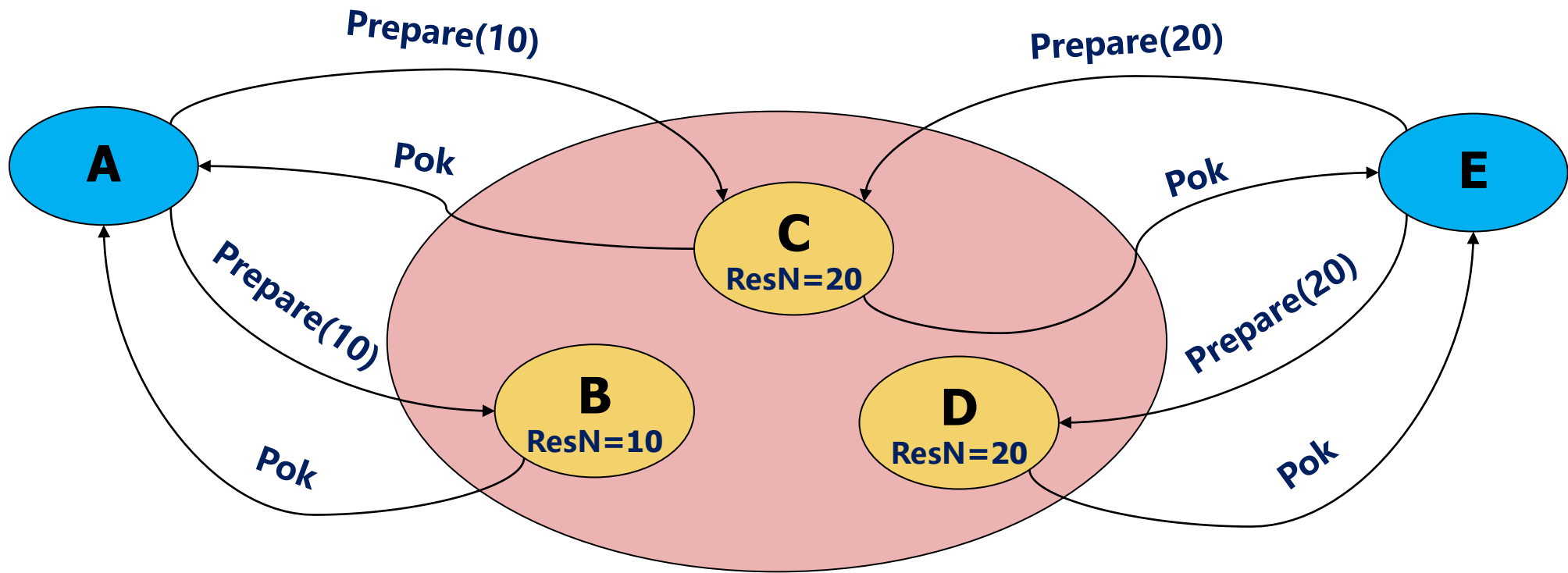
Accept请求

若 $N \geq \text{ResN}$, 接受提案, 令 $\text{AcceptN} = N$,
 $\text{AcceptV} = V$, 回复 <Aok>
若 $N < \text{ResN}$, 不响应或响应 <error>

Accept

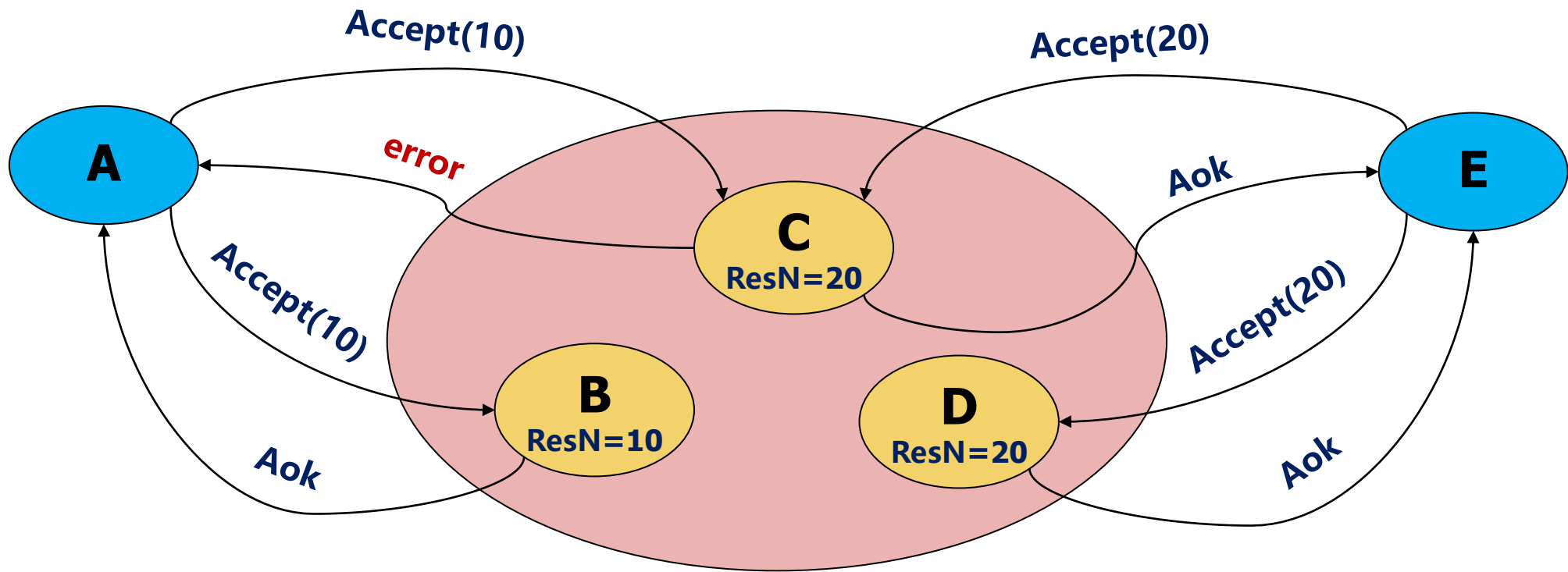
若 <Aok> 数过半, 确定V被选定
若 <Aok> 未超半数, 重新发起Prepare请求

Paxos示例：Prepare阶段



- B先收到A提议，回复A同意；D先收到E提议，回复E同意
- C先收到A提议，回复A同意；再收到E提议后，发现E提议的版本高于A的提议版本，回复E同意

Paxos示例：Accept阶段



- AE分别收到BC和CD回复后，均确定提议被大多数人接受，进入Accept阶段
- C收到A的批准提议后，通知A已有更高版本的提议提出，A随之放弃自己的提议